

Gigafizz part 2: fizz buzz over a slice

CS 551: Systems Programming

Contents

Getting started	1
Introduction	1
What to do	2
Checks to run before you submit	3
Grading	4
Academic honesty	4
Use the toolchain	4

Prerequisite lectures. Through Lecture 6 (slices).

Due. Tuesday 8 September 2026, 11:59PM.

Read `README.md` first. The standing rules there apply. Read `ERRATA.md` before you submit.

This part builds on part 1. The listings below assume `simple_fizzbuzz_no_if_statements` already exists in your `core` crate. If you did not finish part 1, finish it first. You will not get part 1's points back, but you cannot do part 2 without it.

Getting started

You already have the repository. Gigafizz is one repository for the whole semester, and part 2 adds to the one you accepted for part 1.

If you never accepted it, do that first:

<https://classroom50.org/2026-Fall-CS-551/cs-551/assignments/gigafizz/accept?k=zl5swwh0>

That is the same link part 1 used, and it gives you the same repository. Do not accept a second time and do not create a repository by hand.

Introduction

Read the entirety of this document before you start. Deviations from it lose points.

If you have a question about this document, or you think something in it is ambiguous or wrong, ask. Quote the text you are asking about.

What to do

So far Gigafizz computes fizz buzz for one number at a time. Now it does a whole sequence.

The interesting part is not the loop. It is deciding **what type your function takes**, and this part does not tell you. You get the call sites. Work backwards.

Step 1: take the listings

Replace the use line at the top of `cli/src/commands/simple.rs` with:

```
use gigafizz_core::{fizzbuzz_slice, simple_fizzbuzz, simple_fizzbuzz_no_if_statements};
```

Replace `SimpleOptions` with Listing 1:

```
// Listing 1
#[derive(Args, Debug)]
pub(crate) struct SimpleOptions {
    /// What number to calculate fizz buzz on.
    #[clap(
        long,
        short,
        env = "SIMPLE_ON",
        default_value = "100",
        allow_negative_numbers(true)
    )]
    on: i64,

    /// Use the "no if statements" implementation.
    #[clap(long, short)]
    no_if_statements: bool,

    /// Use the slice implementation.
    #[clap(long, short = 'S', conflicts_with = "no_if_statements")]
    use_slice: bool,
}
```

Replace `simple` with Listing 2:

```
// Listing 2
pub(crate) fn simple(opts: &SimpleOptions) {
    tracing::debug!(?opts, "running simple");
    if opts.no_if_statements {
        println!(
            "simple_fizzbuzz_no_if_statements({}): {}",
            opts.on,
            simple_fizzbuzz_no_if_statements(opts.on)
        );
    } else if opts.use_slice {
        let numbers: Vec<i64> = (1..=opts.on).collect();

        let all = fizzbuzz_slice(&numbers);
        println!("all: {} results", all.len());

        let tail = fizzbuzz_slice(&numbers[numbers.len() / 2..]);
        println!("tail: {tail:?}");
    }
}
```

```

    let literal = fizzbuzz_slice(&[3, 5, 15]);
    println!("literal: {literal:?}");

    println!("numbers still has {} elements", numbers.len());
} else {
    println!("simple_fizzbuzz({}): {}", opts.on, simple_fizzbuzz(opts.on));
}
}

```

As always, these are the only changes you may make under `cli/`.

Step 2: write `fizzbuzz_slice`

Add a public function named `fizzbuzz_slice` to `core`. It computes fizz buzz for every number it is given, in order, using the same rules as `simple_fizzbuzz`.

This assignment does not give you the signature. Listing 2 calls `fizzbuzz_slice` three times, and those three calls, taken together, admit exactly one reasonable parameter type. Read them:

- One call passes a `Vec<i64>`.
- One call passes part of that `Vec`, not the whole thing.
- One call passes an array that is not a `Vec` at all.

And read what happens afterwards: `numbers` is still used on the last line, so whatever your function does, it does not consume what it was given.

Pick the type that satisfies all four of those facts at once. If you pick a narrower one, the project will not compile, and the compiler will tell you which call site you broke. That error message is part of the assignment; read it before you change anything.

Do not reimplement the fizz buzz rules. You already have a function that computes them for one number.

Step 3: document it

Your function carries a rustdoc comment with an `# Examples` section that passes as a doctest. **One of your examples must cover the empty case**, that is, what your function returns when it is given nothing at all.

Step 4: update your `README.md`

Add a section named `## Part 2` containing:

1. One sentence saying what type you chose for the parameter.
2. One or two sentences saying why the narrower alternatives do not work. Name at least one specific call site from Listing 2 that would break.

Checks to run before you submit

```

cargo clippy --all-targets    # fix what it reports
cargo fmt                    # then confirm `cargo fmt --check` is silent
cargo test

```

Grading

Out of 100.

Zero on this part if any of these are true:

- Anything under `cli/` is modified beyond the changes in Step 1.
- The workspace does not compile.
- The binary is not named `gigafizz`.
- Your repository has no `HONESTY.md` containing the academic honesty statement from the syllabus.
- `target/` is committed to the repository.

Otherwise:

Points	Item
30	<code>--use-slice</code> produces correct results for all three calls in Listing 2.
20	The parameter type accepts all three call sites and borrows rather than consumes.
10	The function is named <code>fizzbuzz_slice</code> and is public.
10	The implementation reuses your existing single-number function rather than restating the rules.
10	Rustdoc with an <code># Examples</code> section that passes as a doctest.
5	One of those examples covers the empty case.
5	<code>cargo clippy --all-targets</code> is silent, with no <code>#[allow(...)]</code> .
5	<code>cargo fmt</code> produces no diff.
5	<code>README.md</code> has a <code>## Part 2</code> section with both required items.

Academic honesty

Your repository must contain a `HONESTY.md` with the academic honesty statement from the syllabus, followed by your name.

This course does not permit the use of generative AI tools, for code, for documentation, or for correspondence. See the syllabus.

Use the toolchain

The prohibition is on generative AI, not on tooling. **You are encouraged to use the Rust toolchain as fully as you can.** Cargo does a great deal more than `build`, `test`, and `run`, and finding out what it does belongs in this course:

- `cargo clippy --fix` applies the suggestions it can apply mechanically, and implies `--no-deps` and `--all-targets`. It is not a button that empties the warning list: only suggestions marked machine-applicable get rewritten, some lints have no automatic fix at all, and clearing one can uncover another underneath it. Run it, read what it changed, then finish the list by hand. It also refuses to run with uncommitted changes in the tree, which is good advice rather than an obstacle: commit first, so you can see what it did.
- `cargo doc --no-deps --open` renders your documentation the way a reader meets it, which is the fastest way to notice a public item you never documented.
- `cargo tree` shows what your dependencies dragged in behind them.

- `cargo add` edits `Cargo.toml` for you, and records the version it picked.

Ask before you rely on a cargo extension. The subcommands you install with `cargo install cargo-<something>` are interesting and mostly welcome, but name the one you want and say what you want it for. A tool that writes the code for you is still prohibited however it is packaged.